

Kubernetes and Cloud Native Associate - KCNA

Kubernetes Fundamentals

Docker vs ContainerD

In this lesson, we explore the distinctions between Docker and ContainerD, along with several associated CLI tools. As containerization and Kubernetes evolve, you may find older documentation referring to Docker, while newer resources emphasize ContainerD. You might also encounter CLI tools such as CTR, nerdctl, and crictl, and wonder which one best fits your use case. This lesson aims to clarify these differences and outline their respective roles.

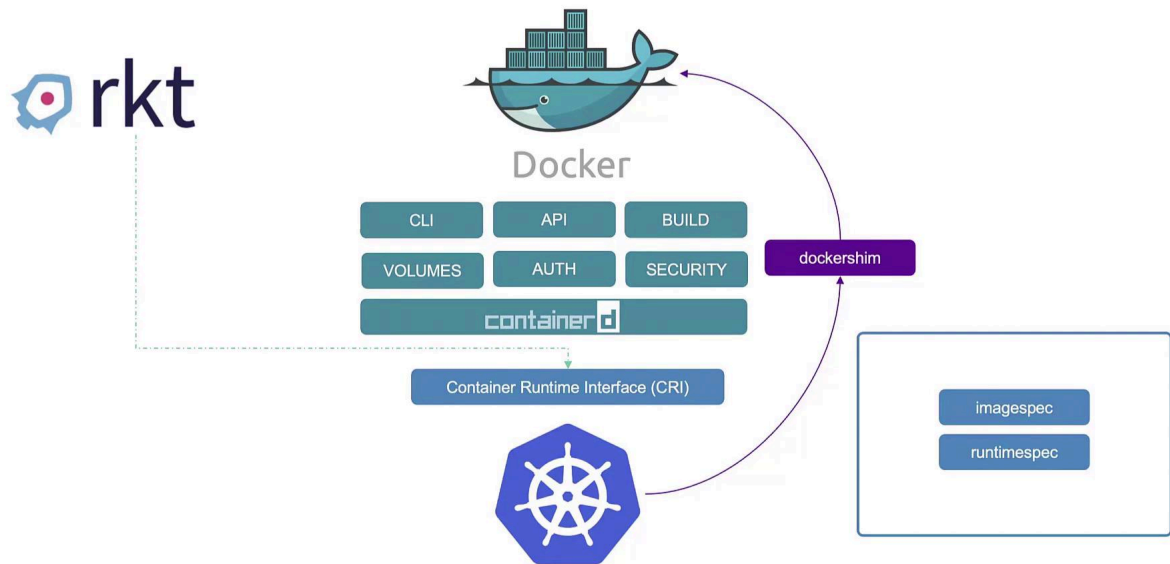
The Evolution of Container Runtimes

Originally, Docker was the dominant tool in the container ecosystem due to its user-friendly approach to container management. When Kubernetes was initially developed, it was tightly integrated with Docker, which led to Docker's pervasive use in container orchestration.

As Kubernetes expanded, alternative container runtimes like Rocket emerged. To address this diversity, the Kubernetes community introduced the Container Runtime Interface (CRI)—an API standard that enables any runtime adhering to the Open Container Initiative (OCI) standards to integrate with Kubernetes. The OCI specifies:

- How images should be built (image specification)
- Standards for container runtimes (runtime specification)

Although Docker images conform to OCI standards, Docker itself predates CRI and lacks native CRI support. To bridge this gap, Kubernetes implemented the Docker Shim, which allowed Kubernetes to interact with Docker despite its non-compliance with CRI. With the maturation of alternatives like ContainerD (which evolved from Docker's architecture), Kubernetes now directly supports these CRI-compatible runtimes, thereby eliminating the extra maintenance overhead of the Docker shim.



Understanding Docker's Components

Docker is more than just a container runtime; it is a suite of tools that includes the Docker CLI, API, build utilities, volume management, authentication, and security features. One important component is containerd—a daemon responsible for container execution using the runc runtime. Although containerd was originally embedded within Docker, it is now maintained as a separate, CRI-compliant project. This separation allows you to install ContainerD independently if you do not require Docker's additional features.

Using ContainerD and Its CLI Tools

Historically, containers were managed using the Docker run command. When running ContainerD as a standalone tool, you can install it with a simple command:

```
$ tar -C /usr/local -xzf containerd-1.6.2-linux-amd64.tar.gz
```

This installation extracts several binaries:

- containerd
- containerd-shim
- containerd-shim-runc-v1
- containerd-shim-runc-v2
- ctr
- containerd-stress

Once installed, ContainerD comes with the command-line tool CTR. Keep in mind that ctr was designed primarily for debugging ContainerD and only supports a limited range of features. For most container management tasks, you will interact with the ContainerD API, which is less user-friendly than a dedicated CLI tool.

For example, to pull a Redis image and run a container using ctr, you can execute:

```
$ ctr images pull docker.io/library/redis:alpine  
$ ctr run docker.io/library/redis:alpine redis
```



Note

For day-to-day container management in production environments, consider using tools that provide a more comprehensive and user-friendly experience.

nerdctl: A Docker-like CLI for ContainerD

The nerdctl tool, developed by the ContainerD community, offers a Docker-like command-line experience while incorporating advanced features such as:

- Encrypted images
- Lazy image pulling
- Peer-to-peer image distribution
- Image signing and verification
- Kubernetes namespace support

Using nerdctl is straightforward. Simply replace the Docker command with nerdctl. For instance, to run a container and map ports, you can use:

```
$ nerdctl run --name redis redis:alpine
$ nerdctl run --name webserver -p 80:80 -d nginx
```

CLI - nerdctl

- nerdctl provides a Docker-like CLI for containerD
- nerdctl supports docker compose
- nerdctl supports newest features in containerd
 - Encrypted container images
 - Lazy Pulling
 - P2P image distribution
 - Image signing and verifying
 - Namespaces in Kubernetes

This tool not only mirrors Docker's functionality but also provides access to innovative features emerging within ContainerD.

The Role of crictl for CRI-Compatible Container Runtimes

crictl is another essential CLI utility designed to interact with CRI-compatible container runtimes from a Kubernetes perspective. Unlike `ctr` and `nerdctl`, which are ContainerD-focused, `crictl` offers a standardized approach that works across all CRI-supported runtimes—including ContainerD, Rocket, and CRI-O.

Maintained by the Kubernetes community, `crictl` is primarily used for inspecting and debugging container runtimes. Although it technically supports container creation, it is not recommended, as containers created outside the control of kubelet (the Kubernetes node agent) may be deleted.

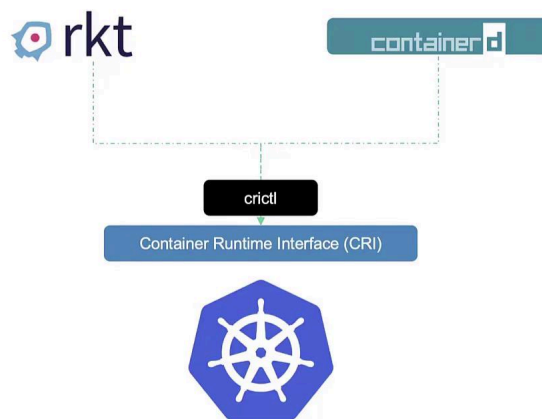
Basic `crictl` commands include:

```
$ crictl pull busybox
$ crictl images
$ crictl ps -a
```

To run commands inside a container or to view logs, `crictl` provides syntax similar to Docker's, while also handling pod-specific details—a feature Docker does not offer.

CLI - crictl

- `crictl` provides a CLI for CRI compatible container runtimes
- Installed separately
- Used to inspect and debug container runtimes
 - Not to create containers ideally.
- Works across different runtimes



A comparison of common Docker CLI commands versus crictl commands highlights many similarities in functionality, covering tasks such as attach, exec, images, info, inspect, logs, ps, stats, and version.

docker vs crictl

Retrieve debugging information [↗](#)

docker cli	crictl	Description	Unsupported Features
attach	attach	Attach to a running container	<code>--detach-keys</code> , <code>--sig-proxy</code>
exec	exec	Run a command in a running container	<code>--privileged</code> , <code>--user</code> , <code>--detach-keys</code>
images	images	List images	
info	info	Display system-wide information	
inspect	inspect , inspecti	Return low-level information on a container, image or task	
logs	logs	Fetch the logs of a container	<code>--details</code>
ps	ps	List containers	
stats	stats	Display a live stream of container(s) resource usage statistics	Column: NET/BLOCK I/O, PIDs
version	version	Show the runtime (Docker, ContainerD, or others) version information	

<https://kubernetes.io/docs/reference/tools/map-crictl-dockercli/>

docker vs crictl

Perform Changes

docker cli	crictl	Description	Unsupported Features
create	create	Create a new container	
kill	stop (timeout = 0)	Kill one or more running container	<code>--signal</code>
pull	pull	Pull an image or a repository from a registry	<code>--all-tags</code> , <code>--disable-content-trust</code>
rm	rm	Remove one or more containers	
rmi	rmi	Remove one or more images	
run	run	Run a command in a new container	
start	start	Start one or more stopped containers	<code>--detach-keys</code>
stop	stop	Stop one or more running containers	
update	update	Update configuration of one or more containers	<code>--restart</code> , <code>--blkio-weight</code> and some other resource limit not supported by CRI.

<https://kubernetes.io/docs/reference/tools/map-crictl-dockercli/>

For additional details on these differences, refer to the documentation linked in the original repository.

Changes in CRI Endpoints with Kubernetes 1.24

In earlier versions of Kubernetes, crictl connected to runtime endpoints using a default order that included several paths:

```
unix:///var/run/dockershim.sock
unix:///run/containerd/containerd.sock
unix:///run/crio/crio.sock
unix:///var/run/cri-dockerd.sock
```

With Kubernetes 1.24, the Docker shim was removed, and the default endpoint configuration was updated. The docker-socket has now been replaced, and users are encouraged to set the runtime endpoint manually:

```
$ crictl --runtime-endpoint
$ export CONTAINER_RUNTIME_ENDPOINT=<your_runtime_endpoint>
```

For example, valid endpoints include:

- `unix:///run/containerd/containerd.sock`
- `unix:///run/crio/crio.sock`
- `unix:///var/run/cri-dockerd.sock`

Reviewing pull request 869 and issue 868 in the Kubernetes CRI tools repository can provide further insights into these changes.

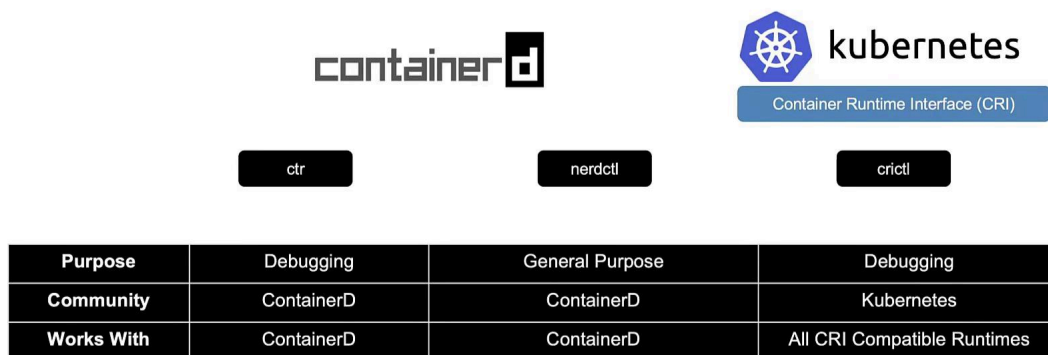
Warning

Always ensure your runtime endpoint is configured correctly to avoid connectivity issues between Kubernetes and your container runtime.

Summary

To recap the key points:

- The CTR tool bundled with ContainerD is intended primarily for debugging, offering limited functionality.
- `nerdctl` provides a full-featured, Docker-like CLI for efficient container management with ContainerD and is ideal for production scenarios.
- `crictl`, maintained by the Kubernetes community, is excellent for debugging and inspecting container runtimes across any CRI-supported platform.



Thank you for engaging with this lesson. We hope it has clarified the differences between Docker, ContainerD, and the various CLI tools used in managing containers and orchestrating Kubernetes deployments.

For more detailed information, please visit the [Kubernetes Documentation](#) or explore related articles on container management.



Watch Video

Watch video content

Previous

◀ **Runtime CRI**

Next

Pods ▶